# Guitar Hero ICMC

Documentação Técnica

Jogo de ritmo em Assembly do Processador ICMC, instrução SNDN, simulador, ferramentas e testes

SSC0511 - Organização de Computadores Digitais - ICMC/USP
Prof. Eduardo do Valle Simões

# Sumário

**1. Visão geral** **4**

1.1 Componentes do repositório . . . . 4

1.2 Fluxo de construção . . . . 4

**2. O Processador ICMC: o que o jogo usa** **6**

2.1 Registradores, memória e pilha . . . . 6

2.2 Diretivas do montador usadas . . . . 6

2.3 Detalhes da semântica que o código respeita . . . . 6

2.4 Vídeo: posição, caractere e cor . . . . 6

**3. Codificação dos dados do jogo** **8**

3.1 Sprites no charmap . . . . 8

3.2 Cenários: letras minúsculas como códigos de sprite . . . . 8

3.3 Mapa da música: uma palavra por frame . . . . 9

3.4 Tabela de frequências . . . . 10

3.5 Tabelas de músicas do menu . . . . 10

3.6 Outras tabelas . . . . 10

3.7 Variáveis de estado . . . . 11

3.8 Notas ativas: 16 slots em vetores paralelos . . . . 11

**4. Tempo, sincronia e janela de acerto** **12**

4.1 O frame do jogo . . . . 12

4.2 Medindo tempo sem relógio: clock fixo de 5 MHz . . . . 12

4.3 Atraso de 22 frames entre o surgimento e o som . . . . 12

4.4 Janela de acerto . . . . 13

4.5 Animação em meia linha . . . . 13

**5. Programa principal e laço do jogo** **14**

**6. Referência das subrotinas** **15**

6.1 Menu e inicialização . . . . 15

6.2 Notas . . . . 16

6.3 Som . . . . 17

6.4 Tempo e teclado . . . . 17

6.5 Acertos, erros e pontuação . . . . 17

6.6 Tela de fim . . . . 19

6.7 Utilitários de desenho . . . . 19

**7. Pontuação: exemplo completo** **21**

**8. A instrução nova: SNDN** **22**

8.1 O problema . . . 22
8.2 Especificação . . . 22
8.3 Decisões de projeto . . . 22
8.4 Implementação no simulador . . . 23
8.5 Montador . . . 23

**9. Outras alterações no simulador** **24**

9.1 Clock fixo de 5 MHz . . . 24
9.2 Correções e melhorias de áudio . . . 24

**10. Ferramentas (tools/)** **25**

10.1 gera_telas.py . . . 25
10.2 gera_musica.py . . . 25
10.3 gera_charmap.py . . . 26
10.4 versao_sound.py e mede_sincronia.py . . . 26

**11. Testes automáticos** **27**

11.1 Harness headless . . . 27
11.2 Cenários do testa_jogo.py . . . 27

**12. Simple Simulator** **29**

12.1 Estrutura de um ciclo . . . 29
12.2 Máquina de estados . . . 29
12.3 Testes do Simple Simulator . . . 30
12.4 Limitações conhecidas da ULA do Simple Simulator . . . 30

**13. Como compilar e executar** **31**

**14. Limites e restrições do projeto** **32**

# 1. Visão geral

O **Guitar Hero ICMC** é um jogo de ritmo escrito inteiramente em Assembly do Processador ICMC, o processador didático de 16 bits usado na disciplina SSC0511 (Organização de Computadores Digitais). Notas descem por quatro pistas e o jogador aperta a tecla da pista (D, F, J ou K) no instante em que a nota passa sobre o alvo. A música toca sozinha, sincronizada com as notas na tela.

Além do jogo, o projeto faz uma **modificação na arquitetura**: uma instrução nova, `SNDN` (som não bloqueante). Ela foi adicionada ao montador e ao simulador porque a instrução de som original (`SOUND`) congela a CPU enquanto a nota toca, o que inviabiliza um jogo de ritmo. O projeto também traz o **Simple Simulator**, uma implementação ciclo a ciclo do processador em C, além de ferramentas em Python que geram partes do código e testes automáticos.

Este documento descreve cada parte do projeto: o que faz, como funciona e por que foi feita assim. Os nomes de rótulos, variáveis e arquivos são os do código-fonte.

## 1.1 Componentes do repositório

| Caminho | Conteúdo |
|---|---|
| `jogo.asm` | o jogo, em Assembly do Processador ICMC (cerca de 2080 linhas) |
| `cpuram.mif` | o jogo montado: imagem da memória de programa/dados (32768 palavras de 16 bits) |
| `charmap.mif` | o mapa de caracteres (fonte 8x8) com os sprites do jogo nos códigos 1 a 26 |
| `Montador/` | montador do Processador ICMC, com a instrução `SNDN` adicionada |
| `Simulador/` | simulador gráfico (GTK2 + SDL2) com `SNDN`, clock fixo e correções de áudio |
| `SimpleSimulator/` | processador ICMC ciclo a ciclo em C (máquina de estados e muxes) |
| `tools/telas/*.txt` | desenhos das telas (jogo, menu, fim de jogo) em texto |
| `tools/musicas/*.txt` | partituras das músicas em formato de texto próprio |
| `tools/gera_telas.py` | converte os desenhos de tela em diretivas `string` no `jogo.asm` |
| `tools/gera_musica.py` | converte as partituras em mapas de notas e na tabela de frequências |
| `tools/gera_charmap.py` | desenha os sprites do jogo dentro do `charmap.mif` |
| `tools/versao_sound.py` | gera a versão "antes" do jogo (com `SOUND`) para comparação |
| `tools/mede_sincronia.py` | mede a sincronia entre a música e os frames do jogo |
| `tools/testa_jogo.py` | testes automáticos de ponta a ponta |
| `tools/headless/` | simulador sem janela, usado pelos testes (mesmo `Model.cpp` do gráfico) |
| `testes/` | programas de teste (`calibra.asm`, `teste_som.asm`, `teste_sndn.asm`, `simples/`) |
| `build.sh` | compila montador, simulador, Simple Simulator e harness |

## 1.2 Fluxo de construção

O código Assembly tem partes **escritas à mão** (a lógica do jogo) e partes **geradas** (telas, músicas, tabela de frequências). As partes geradas ficam entre marcadores de comentário (`; >>> TELA telaL` ... `; <<< TELA telaL` e `; >>> MUSICA` ... `; <<< MUSICA`), e os scripts substituem apenas o que está entre esses marcadores. Assim é possível editar uma tela ou uma música sem tocar no restante do arquivo, e a lógica do jogo nunca é sobrescrita.

```
tools/telas/*.txt   --gera_telas.py-->  \
tools/musicas/*.txt --gera_musica.py--> -+-> jogo.asm --montador--> cpuram.mif --+
                                                                                 +--> Simulador/sim
charmap original    --gera_charmap.py-------------------------> charmap.mif ---+
```

**Por que gerar código?** Desenhar uma tela de 40x30 caracteres ou transcrever uma música diretamente em diretivas `static` é lento e propenso a erro: um caractere a mais desalinha a tela inteira e uma conta errada de bits produz a nota errada. Os scripts validam as entradas (largura das linhas, faixa das notas, notas impossíveis de jogar) antes de escrever qualquer coisa.

# 2. O Processador ICMC: o que o jogo usa

## 2.1 Registradores, memória e pilha

O processador tem oito registradores de uso geral de 16 bits (`r0` a `r7`), o contador de programa `PC`, o ponteiro de pilha `SP`, o registrador de instrução `IR` e o registrador de flags `FR`. A memória tem 32768 palavras de 16 bits e guarda ao mesmo tempo programa e dados (arquitetura de Von Neumann). A pilha começa no fim da memória (`SP = 32767`) e cresce para baixo: `PUSH` grava em `M[SP]` e decrementa `SP`; `POP` incrementa `SP` e lê.

`CALL` grava o endereço de retorno na pilha e salta; `RTS` recupera esse endereço. Como os registradores são poucos, o jogo usa a convenção de que **toda subrotina salva com** `push` **e restaura com** `pop` **todos os registradores que usa**, exceto os de retorno, que são documentados no cabeçalho de cada rotina. Com isso, qualquer rotina pode chamar qualquer outra sem se preocupar com registradores destruídos.

## 2.2 Diretivas do montador usadas

| Diretiva | Efeito |
|---|---|
| `rotulo : var #N` | reserva N palavras na memória a partir de `rotulo` (valor inicial 0) |
| `static rotulo + #i, #v` | define o valor inicial da palavra `rotulo + i` como `v` |
| `rotulo : string "..."` | grava um caractere por palavra e um `\0` (zero) no fim |
| `#rotulo` | o endereço do rótulo, usado como constante (ex.: `loadn r1, #telaL00`) |
| `#'d'` | o código ASCII do caractere (ex.: `loadn r1, #'d'` carrega 100) |

A primeira instrução do programa é `jmp main`, porque o processador começa a executar no endereço 0 e as variáveis e tabelas são declaradas antes do código.

## 2.3 Detalhes da semântica que o código respeita

Alguns comportamentos do simulador não são óbvios e moldaram o código. Eles estão anotados no cabeçalho do `jogo.asm`:

- `CMP` **não altera o flag de zero.** Ela altera só os flags maior/menor/igual. Depois de `CMP` usa-se `jeq`, `jne`, `jgr`, `jle`, `jeg` ou `jel`, nunca `jz`/`jnz`.
- `jle` **é "menor estrito".** Na nomenclatura do ICMC: `jgr` = maior, `jle` = menor (estrito), `jeg` = maior ou igual, `jel` = menor ou igual. Por isso `cmp r0, r1` / `jle X` salta quando `r0 < r1`.
- `INC`**,** `DEC`**,** `ADD` **e as demais operações da ULA atualizam o flag de zero.** `STORE` **e** `STOREI` **não mexem nos flags.** O código usa isso no padrão `dec r0` / `store var, r0` / `jnz laco`: o `jnz` testa o resultado do `DEC`, mesmo com o `STORE` no meio.
- `SUB` **com resultado negativo vira 0** e liga o flag de negativo. O jogo nunca depende de números negativos: todas as contas são feitas de forma que o resultado seja não negativo.
- `INCHAR` **devolve 255 quando nenhuma tecla foi apertada.** O controlador do simulador devolve cada tecla uma única vez e volta a 255, então uma tecla apertada é processada uma única vez.

## 2.4 Vídeo: posição, caractere e cor

A tela tem 40 colunas por 30 linhas, ou seja, 1200 posições. A posição de um caractere é `posicao = coluna + 40 * linha`. A instrução `OUTCHAR Rx, Ry` escreve o caractere `Rx` na posição `Ry`.

A palavra escrita pelo `OUTCHAR` codifica caractere e cor ao mesmo tempo:

```
15      12 11      8 7 6           0
```

```
+-----------+---------+-+---------------+
| (ignorado)|   cor   |x| caractere 0-127|
+-----------+---------+-+---------------+
```

O simulador usa `reg[rx] & 0x7f` como código do caractere e os bits 11-8 como índice da cor. Para escrever um caractere colorido, **basta somar** `cor * 256` **ao código**. Por exemplo, a letra `D` (código 68) em verde (índice 2, ou seja, 512) é `68 + 512 = 580`. As cores usadas pelo jogo:

| Índice | Valor somado | Cor | Uso no jogo |
|---|---|---|---|
| 0 | 0 | branco | texto comum |
| 2 | 512 | verde | pista D, mensagem QUASE |
| 6 | 1536 | verde-azulado | divisórias das pistas |
| 9 | 2304 | vermelho | pista F, mensagens de erro, conceito D |
| 10 | 2560 | verde-limão | mensagem PERFEITO!, conceito A |
| 11 | 2816 | amarelo | pista J, números do placar, título |
| 12 | 3072 | azul | pista K, conceito B |
| 13 | 3328 | rosa | ícone de nota musical, conceito S |
| 14 | 3584 | água | moldura |

# 3. Codificação dos dados do jogo

## 3.1 Sprites no charmap

O `charmap.mif` define o desenho de cada um dos 128 caracteres como uma matriz de 8x8 pixels: são 8 palavras de 8 bits por caractere, 1024 linhas no total (`WIDTH=8; DEPTH=1024`). O caractere de código `c` ocupa as linhas `8c` a `8c+7`. No charmap original, os códigos 0 a 31 (caracteres de controle) ficam em branco, "livres para edição". O jogo desenha seus sprites nesses códigos, sem alterar letras, números e símbolos:

| Código | Sprite | Uso |
|---|---|---|
| 1, 2, 3 | NOTA_ESQ, NOTA_MEIO, NOTA_DIR | a nota (gema), 3 caracteres de largura |
| 5 | PISTA | divisória vertical tracejada entre as pistas |
| 6, 7, 8 | ALVO_ESQ, ALVO_MEIO, ALVO_DIR | alvo vazado na linha 25, mesmo contorno da nota |
| 9, 10 | BORDA_V, BORDA_H | moldura de linha dupla |
| 11 a 14 | cantos | cantos da moldura |
| 15 | BLOCO | bloco cheio (reserva) |
| 16 | NOTA_MUSICAL | enfeite do título |
| 17 | FAISCA | faísca de acerto no centro do alvo |
| 18, 19, 20 | ALVO_CHEIO_* | alvo preenchido: tecla apertada sem nota |
| 21, 22, 23 | NOTA_*_MEIA_CIMA | metade de cima da nota, desenhada no fundo de uma linha |
| 24, 25, 26 | NOTA_*_MEIA_BAIXO | metade de baixo da nota, desenhada no topo da linha seguinte |

**A cor não faz parte do sprite.** Um único desenho de nota serve para as quatro pistas: a cor é somada no Assembly (`1 + 512` = nota verde, `1 + 2304` = nota vermelha). Isso economiza códigos de caractere, que são poucos (31 livres).

**O código 4 não é usado.** O `View.cpp` do simulador pula blocos com `sym == 32`, e `sym = código * 8`. Portanto o código 4 (4 * 8 = 32) nunca é desenhado. O `gera_charmap.py` recusa sprites com esse código.

**Notas de três caracteres.** Uma nota com um único caractere (8 pixels) seria pequena demais para ler numa pista. Com três caracteres (24 pixels) ela tem formato de gema, com pontas arredondadas (`NOTA_ESQ` e `NOTA_DIR`) e um centro vazado (`NOTA_MEIO`). O alvo usa o mesmo contorno vazado, para que o jogador veja claramente quando a nota encaixa no alvo.

## 3.2 Cenários: letras minúsculas como códigos de sprite

As telas são desenhadas em arquivos de texto (`tools/telas/jogo.txt`, `menu.txt`, `fim.txt`). Para que o desenho seja legível num editor comum, usa-se uma convenção: **texto em MAIÚSCULAS é impresso como texto; cada letra minúscula é um código de sprite**. Por exemplo, a linha do alvo é:

```
v          pefgpijkplmnpoqrp          v
```

Aqui, `v` é a borda vertical, `p` é a divisória da pista, `e f g` é o alvo verde (esquerda, meio, direita), `i j k` o vermelho, `l m n` o amarelo e `o q r` o azul. A legenda completa:

| Letra | Sprite | Letra | Sprite |
|---|---|---|---|
| a b c d | cantos da moldura | h | borda horizontal |
| v | borda vertical | p | divisória da pista |

| Letra | Sprite | Letra | Sprite |
|---|---|---|---|
| s | nota musical (rosa) | e f g | alvo verde |
| i j k | alvo vermelho | l m n | alvo amarelo |
| o q r | alvo azul | w x y z | letras D F J K nas cores das pistas |

A tradução é feita em tempo de execução pela tabela `traduz`, um vetor de 128 palavras indexado pelo código ASCII. `traduz[c]` vale 0 quando o caractere é texto comum; caso contrário, contém a palavra final pronta para o `OUTCHAR` (código do sprite + cor). Exemplos:

```
static traduz + #112, #1541     ; 'p' divisoria da pista  ( 5 + verde-azulado 1536)
static traduz + #101, #518      ; 'e' alvo verde, esquerda ( 6 + verde 512)
static traduz + #119, #580      ; 'w' letra D verde       (68 + verde 512)
```

**Por que uma tabela de 128 posições?** A tradução vira uma única leitura indexada (`loadn r1, #traduz` / `add r1, r1, r2` / `loadi r4, r1`), sem cadeias de comparações. Ela custa 128 palavras de memória, que sobram, e é executada para cada caractere desenhado: o cenário inteiro tem 1200.

Cada linha do cenário vira uma diretiva `string` de exatamente 40 caracteres (`telaL00` a `telaL29`). Como o montador grava um `\0` no fim de cada string, **cada linha ocupa 41 palavras na memória**, e as 30 linhas ficam em sequência. Esse detalhe é usado na fórmula de endereço do `RestauraCenario` (seção 6).

## 3.3 Mapa da música: uma palavra por frame

Cada música é um vetor (`musica0`, `musica1`, ...) com **uma palavra de 16 bits por frame** do jogo. A palavra do frame `f` diz quais pistas recebem nota nesse frame e qual som tocar:

```
 15 14 13        10 9              4 3          0
+-----+------------+----------------+------------+
| 0 0 | duração    | índice da nota | pistas     |
|     | (frames)   | em freqs (0-63)| K J F D    |
+-----+------------+----------------+------------+
```

| Campo | Bits | Significado |
|---|---|---|
| pistas | 3-0 | máscara: bit 0 = pista D, bit 1 = F, bit 2 = J, bit 3 = K |
| índice | 9-4 | índice da nota na tabela `freqs` (0 = nenhum som) |
| duração | 13-10 | duração do som, em frames (1 a 15) |
| 65535 | todos | marcador de fim da música |

**Exemplo decodificado.** A primeira nota da Ode à Alegria é `static musica0 + #0, #6420`:

```
6420 = 6 * 1024 + 276          -> duração = 6 frames
 276 = 17 * 16 + 4             -> índice = 17 (E4, 330 Hz), pistas = 4 = 0100b = pista J
```

E a nota final é `12505 = 12 * 1024 + 13 * 16 + 9`: 12 frames, índice 13 (C4), pistas `1001b` = D e K ao mesmo tempo (um acorde).

**Por que essa codificação?**

- **Uma palavra por frame** permite ler o mapa com um acesso indexado direto (`musicaAtual + frame`), sem ponteiros nem contadores de duração. O custo é memória: frames sem nota valem 0 e não precisam de diretiva `static`, porque o `var` já zera a memória. A Ode tem 385 palavras, o que é pouco diante de 32768.
- **Máscara de pistas** permite acordes (várias pistas no mesmo frame) com um simples AND.
- **Índice em vez de frequência.** Uma frequência precisaria de 13 bits (até 4699 Hz) e não caberia junto com os outros campos. Com o índice de 6 bits, a frequência vem da tabela `freqs`.

- **O marcador 65535 nunca colide com uma palavra válida**, porque os bits 15 e 14 de uma palavra válida são sempre 0 (o maior valor possível é `15 * 1024 + 63 * 16 + 15 = 16383`).
- **Limites:** no máximo 15 frames de som por nota (4 bits) e 63 alturas (C3 a D8, 6 bits). O gerador corta notas mais longas para 15 frames de som; a duração musical continua correta, porque a pausa até a próxima nota é dada pela posição dela no vetor, não pelo campo de duração.

Os campos são extraídos no Assembly com divisão e resto por potências de 2, o que equivale a deslocamentos e máscaras:

```
mod r3, r3, #16   -> pistas           (bits 3-0)
div r2, r2, #16   -> descarta as pistas
mod r0, r2, #64   -> índice da nota   (bits 9-4)
div r2, r2, #64   -> duração          (bits 13-10)
```

## 3.4 Tabela de frequências

`freqs[i]` é a frequência, em Hz, da nota de índice `i`. O índice 1 é C3 e cada índice seguinte sobe um semitom, até o 63 (D8). O índice 0 fica livre para significar "sem som". O gerador calcula as frequências pela afinação temperada, `f = 440 * 2^((n - 69) / 12)`, com `n` o número MIDI da nota (`n = índice + 47`), arredondada para inteiro. Exemplos: `freqs[10] = 220` (A3), `freqs[22] = 440` (A4), `freqs[13] = 262` (C4).

## 3.5 Tabelas de músicas do menu

Como o jogo tem várias músicas, o gerador cria tabelas paralelas indexadas por `musicaSel` (0 = primeira música):

| Tabela | Conteúdo |
|---|---|
| `numMusicas` | quantidade de músicas |
| `tabMusicas[i]` | endereço do mapa da música i (`#musica0`, ...) |
| `tabOndas[i]` | forma de onda (0 seno, 1 quadrada, 2 triangular, 3 dente de serra) |
| `tabNomes[i]` | endereço do nome com 20 caracteres, mostrado no menu |
| `tabPainel[i]` | endereço da 1ª de 3 strings de 9 caracteres, mostradas no painel do jogo |

Os nomes têm sempre 20 caracteres (completados com espaços pelo gerador). Assim, ao trocar de música no menu, o nome novo cobre o anterior por completo, sem precisar apagar a área antes. As três linhas do painel ocupam 10 palavras cada (9 caracteres + `\0`) e ficam em sequência; por isso o código passa de uma linha para a próxima somando 10 ao endereço.

## 3.6 Outras tabelas

| Tabela | Valores | Uso |
|---|---|---|
| `pistaX[4]` | 12, 16, 20, 24 | coluna onde começa cada pista (cada nota ocupa 3 colunas e a divisória 1) |
| `pistaCor[4]` | 512, 2304, 2816, 3072 | verde, vermelho, amarelo, azul (as cores do Guitar Hero) |
| `bitPista[4]` | 1, 2, 4, 8 | bit de cada pista na máscara do mapa |
| `tabMsFrame[1..9]` | 200, 150, 112, 85, 65, 50, 37, 30, 22 | duração de um frame em ms para cada nível de velocidade |
| `msgPerfeito` etc. | strings de 15 caracteres | mensagens da linha 28 (largura exata das pistas) |

## 3.7 Variáveis de estado

| Variável | Significado |
|---|---|
| `velocidade` | nível de velocidade escolhido no menu (1 a 9, padrão 5) |
| `musicaSel` | índice da música escolhida no menu |
| `musicaAtual` | endereço do mapa da música em jogo (`tabMusicas[musicaSel]`) |
| `ondaMelodia` | forma de onda da música em jogo (`tabOndas[musicaSel]`) |
| `pontos`, `combo`, `comboMax`, `mult` | pontuação, acertos seguidos, recorde de combo, multiplicador (1 a 4) |
| `acertos`, `erros`, `totalNotas` | contadores do resultado final |
| `hudSujo` | 1 quando o placar mudou e precisa ser redesenhado |
| `frame` | número do frame desde o início da música |
| `spawnAcabou` | 1 quando todas as notas já surgiram |
| `musicaAcabou` | 1 quando a última nota já chegou ao alvo |
| `fimContador` | frames que faltam para mostrar a tela de fim |
| `msgTimer` | frames até apagar a mensagem da linha 28 |
| `notasNoMeio` | 1 quando as notas estão desenhadas meia linha abaixo |
| `faiscaTimer[4]`, `pressTimer[4]` | frames até apagar a faísca e o alvo preenchido de cada pista |
| `acertoSlot`, `acertoY` | auxiliares da `TentaAcerto` (nota escolhida e linha dela) |
| `notaY[16]`, `notaPista[16]` | as notas ativas na tela (seção 3.8) |

## 3.8 Notas ativas: 16 slots em vetores paralelos

As notas que estão na tela ficam em 16 "slots", em dois vetores paralelos: `notaY[i]` é a linha da nota do slot `i` e `notaPista[i]` é a pista dela (0 a 3). `notaY[i] = 0` **significa slot livre**: como as notas nascem na linha 3 e nunca voltam para a linha 0, o próprio campo de linha serve de indicador de ocupação, sem precisar de um terceiro vetor.

**Por que vetores paralelos e não um vetor de registros?** O Processador ICMC não tem modo de endereçamento com deslocamento: o acesso a `base + i` exige `loadn` + `add` + `loadi`. Com vetores paralelos, o endereço de cada campo é `rótulo + slot`, a forma mais simples possível. Com registros de 2 palavras, seria preciso multiplicar o slot por 2.

**Por que 16 slots?** Uma nota leva 22 frames para ir da linha 3 ao alvo e sai da tela poucos frames depois. Mesmo em músicas densas há no máximo cerca de 8 notas visíveis ao mesmo tempo. Dezesseis dá folga, e percorrer 16 slots por frame custa pouco.

# 4. Tempo, sincronia e janela de acerto

## 4.1 O frame do jogo

O jogo é organizado em **frames**. A cada frame, todas as notas descem uma linha, novas notas podem surgir no topo, a melodia pode tocar uma nota e o teclado é lido durante a espera. A duração do frame em milissegundos depende do nível de velocidade (`tabMsFrame`): 200 ms no nível 1, 65 ms no nível 5 (padrão) e 22 ms no nível 9.

## 4.2 Medindo tempo sem relógio: clock fixo de 5 MHz

O Processador ICMC não tem temporizador nem relógio que um programa possa ler. A única forma de medir tempo é **contar instruções**. Isso só funciona se cada instrução levar sempre o mesmo tempo, mas o simulador original rodava "o mais rápido possível": a velocidade do jogo dependia do computador.

Por isso o simulador foi alterado para rodar com **clock fixo de 5 MHz** (constante `CLOCK_HZ` em `Simulador/Mneumonicos.h`): 5 milhões de instruções por segundo, ou seja, **5000 instruções por milissegundo**, como numa placa real. A seção 9.1 descreve a implementação.

A espera do jogo (`EsperaMs`) tem um laço interno de exatamente 5 instruções quando nenhuma tecla é apertada:

```
EsperaMs_interno:
■inchar r0              ; 1
■cmp r0, r3             ; 2
■jeq EsperaMs_semTecla  ; 3
■call ProcessaTecla
EsperaMs_semTecla:
■dec r1                 ; 4
■jnz EsperaMs_interno   ; 5
```

São 1000 voltas por milissegundo, logo 1000 x 5 = 5000 instruções = 1 ms. **A mesma tabela** `tabMsFrame` **dá o tamanho do Delay e a duração das notas da música**, e por isso o som fica naturalmente sincronizado com a tela. O trabalho do frame (mover notas, desenhar) e as voltas do laço externo somam algumas centenas de instruções, que não são descontadas. A medição com `tools/mede_sincronia.py` mostra que cada nota termina a menos de 0,5% do início da seguinte.

**Por que ler o teclado dentro do laço de espera?** Se o teclado fosse lido só uma vez por frame, uma tecla apertada e solta entre duas leituras poderia se perder. Lendo a cada 5 instruções (1 µs), nenhuma tecla se perde, e ela é tratada na hora (`call ProcessaTecla`), com resolução muito melhor que um frame.

## 4.3 Atraso de 22 frames entre o surgimento e o som

Uma nota nasce na linha 3 e desce uma linha por frame; o alvo fica na linha 25. Portanto, uma nota criada no frame `f` chega ao alvo no frame `f + 22`. O mapa da música é lido **duas vezes** em cada frame:

- `SpawnNotas` lê `mapa[frame]` e cria as notas que estão **nascendo** agora.
- `TocaMelodia` lê `mapa[frame - 22]` e toca o som da nota que está **chegando ao alvo** agora.

Com isso, o som sai exatamente quando a nota passa pelo alvo, que é o momento em que o jogador deve apertar a tecla. É o que faz a música e a jogabilidade parecerem a mesma coisa.

## 4.4 Janela de acerto

| Linha da nota quando a tecla é apertada | Resultado | Pontos base |
|---|---|---|
| 22 ou menos | fora da janela: conta como tecla sem nota | 0 (quebra o combo) |
| 23 | QUASE (2 linhas antes) | 5 |
| 24, 25, 26 | PERFEITO! (alvo com 1 linha de folga) | 10 |
| 27 | QUASE (2 linhas depois) | 5 |
| passou da 27 | ERROU! (nota perdida) | 0 (quebra o combo) |

No nível 5, uma linha corresponde a 65 ms, então o PERFEITO aceita cerca de ±65 ms de erro e o QUASE cerca de ±130 ms. A janela em linhas é fixa; em milissegundos ela encolhe nos níveis mais rápidos, e é isso que torna esses níveis mais difíceis.

**Por que a linha 28 é das mensagens?** A janela vai até a linha 27. As mensagens (PERFEITO!, ERROU!...) ficam na linha 28, logo abaixo da última linha por onde uma nota passa, de modo que uma mensagem nunca é desenhada sobre uma nota e vice-versa.

## 4.5 Animação em meia linha

Uma linha de caractere tem 8 pixels. Descendo uma linha por frame, a nota "pula" 8 pixels de cada vez, o que fica visivelmente aos trancos nos níveis lentos. A solução foi dividir cada frame em duas metades: na metade do `Delay`, a rotina `MeioPasso` redesenha todas as notas **4 pixels mais abaixo**, usando os sprites 21 a 26. Esses sprites são a nota cortada ao meio: a metade de cima da nota aparece no fundo da linha Y (sprites 21 a 23) e a metade de baixo no topo da linha Y+1 (sprites 24 a 26). O `gera_charmap.py` gera esses sprites automaticamente a partir do desenho da nota, deslocando as linhas da matriz.

A **linha lógica** (`notaY`) só muda no `MoveNotas`. A meia linha é apenas visual, e a janela de acerto não é afetada. A variável `notasNoMeio` registra se as notas estão desenhadas na posição deslocada, para que o `ApagaNota` saiba que precisa apagar duas linhas em vez de uma. Notas na linha 27 não são deslocadas, porque a linha 28 é das mensagens.

# 5. Programa principal e laço do jogo

```
main:
■call Menu             ; tela inicial: escolhe velocidade e música, espera ESPACO
■call IniciaJogo       ; zera o estado e desenha o cenário
LoopJogo:
■call AtualizaEfeitos  ; apaga mensagens, faíscas e alvos preenchidos vencidos
■call MoveNotas        ; desce as notas uma linha; nota que passou = erro
■call SpawnNotas       ; cria as notas deste frame no topo das pistas
■call TocaMelodia      ; toca a nota que está chegando ao alvo (SNDN)
■call AtualizaHUD      ; redesenha o placar, se mudou
■call Delay            ; espera o tempo do frame lendo o teclado
■frame++
■se musicaAcabou: fimContador--; quando chegar a 0 -> TelaFim; jmp main
```

**Por que esta ordem?**

- `AtualizaEfeitos` vem **antes** de `MoveNotas`. Quando uma faísca vence, o centro do alvo é restaurado a partir do cenário. Se isso acontecesse depois de a nota ser desenhada sobre o alvo, a restauração cobriria a nota. Restaurando primeiro e desenhando as notas depois, as notas ficam sempre por cima.
- `MoveNotas` vem **antes** de `SpawnNotas`. Assim as notas recém-criadas (na linha 3) não são movidas no mesmo frame em que nascem.
- `TocaMelodia` vem depois do movimento, para que o som saia no mesmo frame em que a nota é desenhada na linha 25.
- `AtualizaHUD` vem depois de tudo que pode alterar o placar no frame (notas perdidas no `MoveNotas`).
- `Delay` fica no fim. As teclas são processadas durante ele, e o placar alterado por um acerto é redesenhado no início do frame seguinte, graças ao `hudSujo`.

**Encerramento.** Quando o `TocaMelodia` encontra o marcador 65535 em `mapa[frame - 22]`, a última nota já passou pelo alvo. Ele liga `musicaAcabou` e põe `fimContador = 4`: mais quatro frames para as últimas notas saírem da janela (e serem contadas como erro, se for o caso) antes da tela de fim. O teste do fim usa `dec` / `store` / `jnz`, aproveitando que `STORE` não altera o flag de zero. Depois da `TelaFim`, `jmp main` volta ao menu; `velocidade` e `musicaSel` são mantidas, e todo o resto é zerado pela `IniciaJogo`.

# 6. Referência das subrotinas

Esta seção descreve todas as subrotinas do `jogo.asm`, na ordem em que aparecem no arquivo. "Entrada" e "Saída" indicam registradores; todos os demais registradores são preservados.

## 6.1 Menu e inicialização

### Menu

Desenha a tela inicial (`menuL00` com `ImprimeCenario`), o título colorido (`DesenhaTitulo` na posição 132 = linha 3, coluna 12), o nível de velocidade e o nome da música. Depois fica num laço lendo o teclado com `INCHAR`:

- **ESPAÇO**: sai do menu e o jogo começa.
- **M** ou **m**: `musicaSel = (musicaSel + 1) mod numMusicas`, para avançar circularmente pela lista, e redesenha o nome.
- **1 a 9**: `velocidade = tecla - '0'` e redesenha o número. A faixa é validada com `jle` (menor que '1', ignora) e `jgr` (maior que '9', ignora).
- qualquer outra tecla ou 255 (nenhuma tecla): ignora.

O laço fica preso no `INCHAR` sem nenhum atraso, porque no menu não há nada para animar.

### MostraVelocidadeMenu

Escreve `velocidade` com 1 dígito, em amarelo, na posição 855 (linha 21, coluna 15), via `ImprimeNumero`.

### MostraMusicaMenu

Lê `tabNomes[musicaSel]` (endereço do nome) e o escreve em amarelo na posição 891 (linha 22, coluna 11), via `ImprimeStrT`. Os nomes têm sempre 20 caracteres, então o nome anterior é coberto por inteiro.

### DesenhaTitulo

**Entrada:** `r0` = posição. Escreve a string `tituloJogo` ("GUITAR HERO ICMC") em amarelo. É usada no menu e na tela do jogo, porque o cenário é impresso com texto branco e o título deve se destacar.

### IniciaJogo

Prepara uma partida nova:

1. Zera `pontos`, `combo`, `comboMax`, `acertos`, `erros`, `totalNotas`, `frame`, `spawnAcabou`, `musicaAcabou`, `fimContador`, `msgTimer`, `notasNoMeio`, e põe `mult = 1`.
2. Zera os 16 slots de `notaY` (todos livres).
3. Zera `faiscaTimer` e `pressTimer` com um único laço de 8 palavras, porque os dois vetores de 4 palavras foram declarados um logo após o outro e ficam vizinhos na memória.
4. Desenha o cenário do jogo (`telaL00`), o título e o nível de velocidade (posição 769 = linha 19, coluna 9).
5. Copia `tabMusicas[musicaSel]` para `musicaAtual` e `tabOndas[musicaSel]` para `ondaMelodia`. Assim, durante o jogo, as rotinas usam uma variável direta em vez de indexar as tabelas a cada frame.
6. Escreve as 3 linhas do painel com o nome da música (posição 202 = linha 5, coluna 2; a cada linha, +40 na posição e +10 no endereço da string).
7. Liga `hudSujo` e chama `AtualizaHUD` para desenhar o placar zerado.

## 6.2 Notas

### MoveNotas

Percorre os 16 slots. Para cada slot ocupado (`notaY != 0`):

1. `ApagaNota` na posição atual. Isso é feito **antes** de alterar a linha, porque a posição a apagar é calculada a partir de `notaY`.
2. `notaY[slot]++`.
3. Se a nova linha passou de 27 (`jgr`), a nota foi perdida: libera o slot (`notaY = 0`) e chama `Errou`. Senão, chama `DesenhaNota` na nova linha.

No fim, põe `notasNoMeio = 0`, porque todas as notas foram redesenhadas alinhadas.

### SpawnNotas

Se `spawnAcabou` já vale 1, não faz nada. Senão, lê `mapa[frame]` (com `musicaAtual + frame`). Se a palavra é 65535, liga `spawnAcabou`. Caso contrário, extrai a máscara (`mod 16`) e, para cada pista 0 a 3, testa `mascara AND bitPista[pista]`; se o bit está ligado, chama `CriaNota` com `r0 = pista`.

### CriaNota

**Entrada:** `r0` = pista. Procura o primeiro slot livre (`notaY = 0`), põe `notaY = 3` (linha onde as notas nascem) e `notaPista = pista`, incrementa `totalNotas` e desenha a nota. Se não houver slot livre, a nota é descartada em silêncio. Isso não acontece na prática (há no máximo cerca de 8 notas na tela), e descartar é mais seguro do que sobrescrever uma nota ativa.

**Por que nascer na linha 3?** As linhas 0 a 2 são a moldura e o título. A linha 3 é a primeira linha livre das pistas.

### PosicaoNota

**Entrada:** `r0` = slot. **Saída:** `r1` = posição na tela da parte esquerda da nota; `r2` = cor da pista. Calcula `r1 = notaY[slot] * 40 + pistaX[notaPista[slot]]` e `r2 = pistaCor[notaPista[slot]]`. Está centralizada numa rotina porque desenhar, apagar e desenhar em meia linha precisam da mesma conta.

### DesenhaNota

**Entrada:** `r0` = slot. Escreve os sprites 1, 2 e 3 (somados à cor da pista) em três posições consecutivas. Os códigos são consecutivos de propósito: basta um `inc` no caractere e outro na posição entre os `OUTCHAR`.

### ApagaNota

**Entrada:** `r0` = slot. Apaga a nota **redesenhando o cenário por baixo** dela (`RestauraCenario` nas 3 posições). Se `notasNoMeio = 1` e a nota não está na linha 27 ou abaixo (posição >= 1080), apaga também as 3 posições da linha de baixo, onde fica a metade inferior da nota em meia linha.

**Por que restaurar o cenário em vez de escrever espaços?** As notas passam por cima das divisórias, do alvo (linha 25) e das letras das teclas (linha 26). Escrevendo espaço, o alvo e as letras desapareceriam depois da primeira nota. Restaurando o caractere original do cenário, a tela volta exatamente ao que era.

### DesenhaNotaMeia

**Entrada:** `r0` = slot. Desenha a nota deslocada meia linha para baixo: sprites 21, 22 e 23 na linha Y e sprites 24, 25 e 26 na mesma coluna da linha Y+1. A passagem para a linha de baixo soma 38 à posição: +40 de uma linha, -2 para voltar à primeira coluna, já que a posição avançou 2 durante a parte de cima.

### MeioPasso

Chamada pelo `Delay` na metade do frame. Para cada slot ocupado com linha menor que 27: `ApagaNota` (com `notasNoMeio` ainda 0, apaga só a linha Y) e `DesenhaNotaMeia`. No fim, liga `notasNoMeio`.

## 6.3 Som

### TocaMelodia

Toca a nota da música que está chegando ao alvo neste frame:

1. Se `musicaAcabou` vale 1, retorna.
2. Se `frame < 22`, retorna, porque nenhuma nota chegou ainda ao alvo. O teste é `jle`, menor estrito, então no frame 22 o índice é 0.
3. Lê `mapa[frame - 22]`. Se a palavra é 65535: liga `musicaAcabou`, põe `fimContador = 4` e retorna.
4. Extrai o índice (bits 9-4) e a duração (bits 13-10). Índice 0 significa frame sem nota: retorna.
5. `r0 = freqs[índice]` (Hz); `r1 = tabMsFrame[velocidade] * duração` (ms); `r2 = ondaMelodia`.
6. `sndn r0, r1, r2`: **a nova instrução** inicia a nota e devolve o controle à CPU no mesmo ciclo.

A duração em ms usa a mesma tabela do `Delay`, então uma nota de 6 frames dura exatamente o tempo de 6 frames e termina quando a próxima começa. O maior valor possível é 15 frames x 200 ms = 3000 ms, que cabe com folga em 16 bits.

## 6.4 Tempo e teclado

### Delay

Espera a duração de um frame em duas metades: `r1 = ms / 2` e `r2 = ms - r1`, para que o resto da divisão não se perca em números ímpares. Chama `EsperaMs` na primeira metade, `MeioPasso` e `EsperaMs` na segunda.

### EsperaMs

**Entrada:** `r2` = milissegundos (maior que 0). Laço externo de `r2` voltas, cada uma com um laço interno de 1000 voltas de 5 instruções (seção 4.2). Em cada volta interna, lê o teclado e, se houver tecla (valor diferente de 255), chama `ProcessaTecla`.

### ProcessaTecla

**Entrada:** `r0` = código da tecla. Converte `d`/`D` na pista 0, `f`/`F` na 1, `j`/`J` na 2 e `k`/`K` na 3, e chama `TentaAcerto`. Outras teclas são ignoradas. Maiúsculas e minúsculas são aceitas para que o jogo funcione com Caps Lock ligado.

## 6.5 Acertos, erros e pontuação

### TentaAcerto

**Entrada:** `r0` = pista. Procura, entre os 16 slots, as notas da pista com linha >= 23 (dentro da janela) e escolhe a de **maior linha**, ou seja, a mais baixa e mais antiga, a que sairia da janela primeiro. Se duas notas da mesma pista estão na janela, a tecla vale para a que está mais perto de ser perdida, como nos jogos de ritmo comerciais. Se o critério fosse "a primeira no vetor", a ordem dependeria do slot em que cada nota caiu, o que é arbitrário.

O teste `cmp r3, r4` / `jle` (linha < 23) também descarta os slots livres, porque eles têm linha 0. O limite superior não é testado: uma nota com linha maior que 27 já foi removida pelo `MoveNotas`.

Se encontrou uma nota, chama `Acertou` com `r1` = slot e `r2` = linha. Senão, chama `AcendePressionado` (alvo preenchido) e `QuebraCombo` com a mensagem "NADA AQUI!".

**Por que** `acertoSlot` **e** `acertoY` **ficam na memória?** A rotina já usa `r0` (pista) e `r1` a `r7` no laço. Não há registrador livre para guardar a melhor nota encontrada, então ela fica em duas variáveis. O valor 255 em `acertoSlot` significa "nenhuma nota encontrada".

### Acertou

**Entrada:** `r1` = slot, `r2` = linha da nota.

1. Apaga a nota, libera o slot e acende a faísca na pista dela.
2. Linhas 24 a 26: 10 pontos, mensagem "PERFEITO!" em verde-limão. Linha 23 ou 27: 5 pontos, "QUASE" em verde.
3. `pontos += base * mult`, com o multiplicador **de antes** deste acerto.
4. `combo++`; se `combo > comboMax`, atualiza o recorde.
5. `mult = min(1 + combo / 10, 4)`: sobe a cada 10 acertos seguidos, até x4.
6. `acertos++` e `hudSujo = 1`.

### Errou

Chamada pelo `MoveNotas` quando uma nota passa da linha 27. Chama `QuebraCombo` com a mensagem "ERROU!".

### QuebraCombo

**Entrada:** `r1` = mensagem. Zera o combo, põe o multiplicador em 1, incrementa `erros`, liga `hudSujo` e mostra a mensagem em vermelho. É compartilhada por "nota perdida" e "tecla sem nota", porque as duas situações têm a mesma consequência.

### MostraMensagem

**Entrada:** `r1` = mensagem de 15 caracteres, `r2` = cor. Apaga a mensagem anterior, escreve a nova na posição 1132 (linha 28, coluna 12, alinhada com as pistas) e põe `msgTimer = 12`: a mensagem some após 12 frames.

### ApagaMensagem

Restaura o cenário nas posições 1132 a 1146 (linha 28, colunas 12 a 26).

### AcendeFaisca

**Entrada:** `r0` = pista. Se o alvo estava preenchido por uma tecla errada, restaura o alvo e zera o `pressTimer` da pista. Depois desenha o sprite 17 (faísca), na cor da pista, no centro do alvo (posição `pistaX + 1001` = linha 25, coluna central da nota) e põe `faiscaTimer = 4`.

### AcendePressionado

**Entrada:** `r0` = pista. Desenha o alvo preenchido (sprites 18, 19 e 20 na cor da pista) e põe `pressTimer = 3`. Zera o `faiscaTimer` da pista, porque o preenchimento cobre uma faísca antiga. Se o timer da faísca continuasse, ele restauraria o centro do alvo no meio do preenchimento, deixando um buraco.

**Por que mostrar o alvo preenchido?** Sem esse retorno visual, o jogador não saberia se a tecla foi lida e simplesmente não havia nota. O preenchimento deixa claro que a tecla funcionou mas foi apertada na hora errada.

### RestauraAlvo

**Entrada:** `r0` = pista. Redesenha as 3 posições do alvo (linha 25) a partir do cenário.

### AtualizaEfeitos

Executada no início de cada frame. Decrementa os temporizadores e apaga o que venceu:

- `msgTimer`: ao chegar a 0, `ApagaMensagem`.
- `faiscaTimer[p]`: ao chegar a 0, restaura o centro do alvo (`pistaX + 1001`).
- `pressTimer[p]`: ao chegar a 0, `RestauraAlvo`.

Temporizadores já em 0 são ignorados (inativos). O teste usa `dec` / `storei` / `jnz`, porque `STOREI` não altera o flag de zero.

**Por que temporizadores em frames?** Os efeitos precisam durar um tempo visível, mas o jogo não pode parar para esperar. Contar frames reaproveita o relógio do jogo, sem nenhum custo extra.

### AtualizaHUD

Se `hudSujo` vale 0, retorna. Senão, zera `hudSujo` e escreve, em amarelo, `pontos` (5 dígitos, posição 230), `combo` (3 dígitos, 350), `mult` (1 dígito, 471, logo depois do "X"), `acertos` (3 dígitos, 590) e `erros` (3 dígitos, 710).

**Por que o `hudSujo`?** `ImprimeNumero` faz uma divisão e um resto por dígito: são 15 dígitos e centenas de instruções no total. Na maioria dos frames nada muda no placar, e o "flag de sujeira" evita esse trabalho repetido.

## 6.6 Tela de fim

### TelaFim

Desenha a caixa de resultado (`fimL00` a `fimL10`, 22 colunas x 11 linhas) **sobre** a tela do jogo, a partir da posição 369 (linha 9, coluna 9). Cada string da caixa ocupa 23 palavras (22 + `\0`), então o endereço avança 23 e a posição avança 40 por linha. Depois preenche pontos, acertos, total de notas, combo máximo, precisão (`CalculaPrecisao`) e conceito (`Conceito`), e espera ESPAÇO.

### CalculaPrecisao

**Saída:** `r1` = `acertos * 100 / totalNotas`, ou 0 se `totalNotas = 0`, para evitar divisão por zero. A multiplicação vem antes da divisão para não perder precisão na divisão inteira. `acertos * 100` cabe em 16 bits até 655 acertos.

### Conceito

**Saída:** `r1` = letra com cor, pronta para `OUTCHAR`. Calcula a precisão e compara em cascata com `jeg`:

| Precisão | Conceito | Cor |
|---|---|---|
| >= 95% | S | rosa (3411 = 83 + 3328) |
| >= 85% | A | verde-limão |
| >= 70% | B | azul |
| >= 50% | C | amarelo |
| < 50% | D | vermelho |

## 6.7 Utilitários de desenho

### Traduz

**Entrada:** `r2` = caractere, `r3` = cor padrão do texto. **Saída:** `r2` = palavra final. Se `traduz[r2]` é diferente de 0, o caractere é um sprite e o resultado é o valor da tabela (já com a cor). Senão, é texto comum e o resultado é `r2 + r3`.

### ImprimeCenario

**Entrada:** `r1` = endereço da primeira linha (ex.: `#telaL00`), `r3` = cor do texto. Desenha 30 linhas de 40 caracteres, traduzindo cada um. Depois de cada linha, pula o `\0` (`inc r1` extra). O laço termina quando a posição chega a 1200.

### RestauraCenario

**Entrada:** `r0` = posição. Redesenha o caractere do cenário do jogo naquela posição. Como cada linha do cenário ocupa 41 palavras (40 + `\0`), o endereço do caractere é:

```
endereço = telaL00 + linha * 41 + coluna
         = telaL00 + (linha * 40 + coluna) + linha
         = telaL00 + posição + posição / 40
```

Ou seja, basta somar a linha (`posição / 40`) à posição: uma divisão e duas somas, sem multiplicação. É a rotina mais chamada do jogo: ela apaga notas, mensagens, faíscas e alvos.

### ImprimeStrT

**Entrada:** `r0` = posição, `r1` = endereço da string, `r3` = cor. Imprime até o `\0`, traduzindo sprites. O "T" do nome vem de "traduzida": os nomes e as mensagens podem conter sprites, como no título do menu.

### ImprimeNumero

**Entrada:** `r0` = posição do primeiro dígito, `r1` = valor, `r2` = número de dígitos, `r3` = cor. Escreve o número em decimal, com zeros à esquerda, **da direita para a esquerda**: começa na posição do último dígito, escreve `valor mod 10` e faz `valor = valor / 10`, repetindo `r2` vezes. Assim não é preciso calcular potências de 10 nem saber de antemão quantos dígitos o número tem. A cor é somada uma única vez a `'0'` antes do laço (`r3 = cor + '0'`), e cada dígito vira uma soma só.

# 7. Pontuação: exemplo completo

A Ode à Alegria tem 62 frames com nota e 63 notas no total (o acorde final tem duas). Acertando todas como PERFEITO:

| Acertos | Multiplicador usado | Pontos |
|---|---|---|
| 1 a 10 | x1 (o combo só chega a 10 depois do 10º acerto) | 10 x 10 = 100 |
| 11 a 20 | x2 | 10 x 20 = 200 |
| 21 a 30 | x3 | 10 x 30 = 300 |
| 31 a 63 | x4 | 33 x 40 = 1320 |
| **total** | | **1920** |

Acertando todas como QUASE, o total é a metade: 960. Os dois valores são verificados pelos testes automáticos (seção 11). A pontuação cabe em 16 bits enquanto houver menos de cerca de 1600 notas (65535 / 40), e o placar mostra 5 dígitos.

# 8. A instrução nova: SNDN

## 8.1 O problema

O Processador ICMC já tinha uma instrução de som, `SOUND Rx, Ry, Rz`. No simulador, ela chama `play_note`, que coloca a nota na fila de áudio e depois executa `beeper.wait()`: um laço que só termina quando a nota acaba de tocar. Ou seja, **a instrução `SOUND` dura o tempo da nota inteira**. Além disso, no simulador gráfico a CPU executa com o lock da interface (`gdk_threads_enter`), então a janela inteira congela junto.

Num jogo de ritmo isso é fatal: enquanto a nota toca, as notas não descem e o teclado não é lido. A medição com o harness, tocando a música inteira no nível 5:

| Versão | Notas tocadas | CPU travada pelo som |
|---|---|---|
| melodia com `SOUND` (`tools/versao_sound.py`) | 62 | **24 960 ms** (a música inteira) |
| melodia com `SNDN` (`jogo.asm`) | 62 | **0 ms** |

## 8.2 Especificação

| **Mnemônico** | `SNDN Rx, Ry, Rz` |
|---|---|
| Operandos | Rx = frequência (Hz), Ry = duração (ms), Rz = forma de onda (0 seno, 1 quadrada, 2 triangular, 3 dente de serra) |
| Opcode | `110110` (54) |
| Tamanho | 1 palavra |
| Flags | não altera |
| Semântica | interrompe a nota que estiver tocando, inicia a nova e termina no mesmo ciclo |

```
 15      10 9   7 6   4 3   1 0
+----------+-----+-----+-----+-+
|  110110  | Rx  | Ry  | Rz  |0|
+----------+-----+-----+-----+-+

sndn r1, r2, r3  ->  110110 001 010 011 0  =  1101100010100110
```

## 8.3 Decisões de projeto

- **Opcode `110110`.** Estava livre e fica na família de E/S (`1101xx`), ao lado de `SOUND` (`110100`) e `INCHAR` (`110101`). O formato é idêntico ao do `SOUND`, então o decodificador reaproveita os campos Rx, Ry e Rz sem nenhuma mudança. Numa FPGA, a unidade de controle trataria `110110` como o `SOUND`, só que sem o estado de espera.
- **Interromper em vez de enfileirar (síntese monofônica).** Se a `SNDN` enfileirasse as notas, qualquer diferença entre a duração pedida e a duração real de um frame se acumularia, e a música sairia de sincronia com a tela ao longo da partida. Interrompendo, cada nota começa exatamente no ciclo da instrução: **quem dita o ritmo é o jogo**. É o comportamento de um "note on" num sintetizador monofônico.
- **Mesmos operandos do `SOUND`.** Não há conceito novo a aprender, e trocar uma instrução pela outra é uma substituição de texto. O `versao_sound.py` faz exatamente isso para gerar a versão de comparação.
- **Um único dispositivo de áudio.** O original chamava `SDL_Init` e abria o dispositivo de áudio a cada nota. Com duas instruções de som no mesmo programa, dois `SDL_OpenAudio` simultâneos falhariam. Agora há um `Beeper` global, criado na primeira nota e compartilhado por `SOUND` e `SNDN`. **O `SOUND` mantém a semântica original** (bloqueante), para não quebrar programas existentes.

## 8.4 Implementação no simulador

`Simulador/Mneumonicos.h` define o opcode (`#define SNDN 54`). `Simulador/Model.cpp` executa a instrução ao lado do `SOUND`:

```
case SOUND:
  play_note(reg[rx], reg[ry], reg[rz]);         // bloqueante
  break;
case SNDN:    // Rx = frequencia (Hz), Ry = duracao (ms), Rz = forma de onda
  play_note_async(reg[rx], reg[ry], reg[rz]);   // nao espera
  break;
```

Em `Simulador/Sound.cpp`, o áudio é gerado pelo SDL num callback que roda em outra thread: o SDL pede blocos de amostras e o `Beeper::generateSamples` os preenche a partir de uma fila de notas (`queue<BeepObject>`). As duas instruções diferem só no que fazem com essa fila:

```
void Beeper::replace(string wave, double freq, int duration){
    BeepObject bo; ...                      // monta a nota nova
    SDL_LockAudio();                        // exclusão mútua com o callback
    beeps = queue<BeepObject>();            // descarta a nota atual e as pendentes
    beeps.push(bo);
    SDL_UnlockAudio();
}

void play_note (int rx, int ry, int rz) {        // SOUND
    Beeper& beeper = beeperGlobal();
    beeper.beep(tipoOnda(rz), rx, ry);           // enfileira
    beeper.wait(30);                             // espera a fila esvaziar
}

void play_note_async (int rx, int ry, int rz) {  // SNDN
    beeperGlobal().replace(tipoOnda(rz), rx, ry);
}
```

`SDL_LockAudio` / `SDL_UnlockAudio` são necessários porque a fila é lida pelo callback de áudio em outra thread. Sem o lock, a CPU simulada poderia trocar a fila no meio de uma leitura. `beeperGlobal()` usa variáveis `static` locais, que o C++ inicializa uma única vez, na primeira chamada.

## 8.5 Montador

Seguindo o roteiro da disciplina para adicionar instruções, são três pontos no `Montador/defs.h` e três no `Montador/montador.c`:

```
#define SNDN_CODE   99              /* identificador interno     */
#define SNDN        "110110"        /* opcode em binário         */
#define SNDN_STR    "SNDN"          /* mnemônico                 */
```

1. **1ª passada (contagem de palavras):** `case SNDN_CODE:` junto com `SOUND_CODE`, ou seja, 3 registradores e 1 palavra.
2. **2ª passada (geração):** lê os 3 registradores e escreve `sprintf(str_msg, "%s%s%s%s0", SNDN, str_tmp1, str_tmp2, str_tmp3);`.
3. **Reconhecimento do mnemônico:** `else if (strcmp(str_tmp, SNDN_STR) == 0) return SNDN_CODE;`.

O desassemblador do painel "Memória" do simulador (`View.cpp`) também passou a mostrar `SOUND` e `SNDN`. Antes, o `SOUND` caía no `default` e imprimia `ERRO - show program linha` no console.

# 9. Outras alterações no simulador

## 9.1 Clock fixo de 5 MHz

O laço do modo automático (`processaAutomatico`, em `Model.cpp`) executava instruções sem parar e chamava um `delay()` a cada 500 instruções, então a velocidade dependia do computador. A versão nova mede o tempo real:

```
const long long BLOCO = 2000;
gint64 inicio = g_get_monotonic_time();          // microssegundos
long long executadas = 0;
while (model->getProcessamento()) {
    gdk_threads_enter(); model->processador(); gdk_threads_leave();
    if (++executadas % BLOCO == 0) {
        gint64 alvo  = inicio + executadas * 1000000LL / CLOCK_HZ;
        gint64 agora = g_get_monotonic_time();
        if (alvo - agora > 1000)        g_usleep(alvo - agora);   // adiantado: dorme
        else if (agora - alvo > 100000) { inicio = agora; executadas = 0; }  // muito atrasado
        else                            g_thread_yield();
    }
}
```

- **A cada 2000 instruções** (0,4 ms de tempo simulado), compara o tempo que deveria ter passado (`executadas / CLOCK_HZ`) com o tempo real, e dorme a diferença se estiver adiantado mais de 1 ms. Comparar contra o instante inicial, e não contra o último bloco, evita que erros de arredondamento do `sleep` se acumulem.
- **Se estiver atrasado mais de 100 ms** (computador lento ou uma pausa longa num `SOUND`), não tenta compensar acelerando, o que causaria uma rajada de frames: apenas recomeça a contagem.
- **A pausa libera o lock do GTK**, e com isso a janela consegue se redesenhar com regularidade. Antes, a thread da tela quase nunca obtinha o lock, e as notas desciam aos trancos.
- **Windows:** `main.cpp` chama `timeBeginPeriod(1)` (winmm). Sem isso, o Windows só acorda threads a cada ~15 ms, e os frames ficariam irregulares.

O programa `testes/calibra.asm` confere o clock: ele executa blocos de ~20 000 instruções e mostra a contagem na tela. Com 5 MHz devem sair cerca de 250 blocos por segundo.

## 9.2 Correções e melhorias de áudio

| Mudança | Motivo |
|---|---|
| `SDL_OpenAudio(&desired, NULL)` em vez de passar um `obtainedSpec` | com `obtainedSpec`, o SDL2 entrega ao callback o formato nativo da placa, sem conversão. No Windows (WASAPI, float 32 bits estéreo) o callback escrevia inteiros de 16 bits mono e o simulador ficava **mudo**. Com `NULL`, o SDL converte. `testes/diagnostico_audio.cpp` mostra a diferença na máquina |
| buffer de 512 amostras (~12 ms) em vez de 2048 (~46 ms) | com blocos grandes, o som saía visivelmente atrasado em relação à nota chegando ao alvo |
| envelope de 5 ms no início e no fim de cada nota (`RAMPA = FREQUENCY / 200`) | sem rampa, o começo e o fim da onda são degraus e cada nota "estala" |
| `AMPLITUDE = 7000` (era 28000) | quase o máximo de 16 bits: com a onda quadrada ficava alto demais (-12 dB) |
| fase v inicializada com 0 | no original ela começava com lixo de memória |

# 10. Ferramentas (tools/)

## 10.1 gera_telas.py

Converte `tools/telas/*.txt` nas diretivas `string` do `jogo.asm`. Cada arquivo começa com um cabeçalho (`# prefixo telaL` e `# largura 40`).

| Função | O que faz |
|---|---|
| `le_tela(caminho)` | lê o cabeçalho e as linhas; recusa aspas, barra invertida e caracteres fora do ASCII imprimível (o montador não os aceita dentro de `string`); exige que **todas** as linhas tenham exatamente a largura declarada |
| `injeta(asm, prefixo, linhas, origem)` | substitui o bloco entre `; >>> TELA <prefixo>` e `; <<< TELA <prefixo>` por uma linha `<prefixo>NN : string "..."` para cada linha do desenho |
| `main()` | aplica as telas passadas na linha de comando e regrava o `jogo.asm` |

A validação de largura é essencial: o `ImprimeCenario` e o `RestauraCenario` assumem exatamente 40 caracteres por linha, e uma linha com 39 deslocaria todo o resto da tela.

## 10.2 gera_musica.py

Converte as partituras em mapas de notas. Formato resumido (a especificação completa está em `docs/formato_partitura.md`):

```
nome ODE A ALEGRIA              # nome no menu (até 20 caracteres)
painel ODE A|ALEGRIA|BEETHOVEN  # 3 linhas do painel (até 9 cada)
onda 2                          # 0 seno, 1 quadrada, 2 triangular, 3 dente de serra
slots_por_tempo 6               # frames por tempo (semínima)
pistas G3=0 C4=0 D4=1 E4=2 F4=3 G4=3
E4 1                            # NOTA DURAÇÃO_EM_TEMPOS
D4 1.5                          # pontuada
- 1                             # pausa
C4+G4 2                         # acorde: duas notas caem juntas (só a 1ª soa)
```

| Função | O que faz |
|---|---|
| `midi(nome)` | converte "F#4", "Bb3" etc. no número MIDI: `12 * (oitava + 1) + semitom` |
| `nome_midi(n)` | o inverso, para mensagens e comentários |
| `freq_hz(n)` | `round(440 * 2^((n - 69) / 12))` |
| `texto_tela(...)` | valida e converte para maiúsculas os textos de `nome` e `painel`, completando com espaços até a largura fixa |
| `le_partitura(caminho)` | lê diretivas e eventos; as durações são lidas como `Fraction`, então `1/3` (tercinas) é exato |
| `pistas_automaticas(eventos)` | sem linha `pistas`: ordena as alturas usadas e as distribui igualmente entre as 4 pistas, das graves (0) para as agudas (3) |
| `gera_mapa(caminho, info)` | gera a lista de palavras: para cada evento, `frames = tempos * slots_por_tempo` (precisa ser inteiro); monta `mascara \| indice << 4 \| min(frames, 15) << 10` e acrescenta `frames - 1` zeros |
| `bloco_asm(musicas)` | escreve as tabelas do menu, os mapas (com comentário legível em cada palavra) e a tabela `freqs` |
| `valida(caminho)` | modo `--validar`: confere a partitura sem alterar o jogo e aponta trechos injogáveis |
| `main()` | gera todas as músicas passadas, **na ordem dos arquivos**, e substitui o bloco `; >>> MUSICA` |

**Regras de jogabilidade verificadas pelo `--validar`:** duas notas na mesma pista a menos de 2 frames de distância são **erro** (os sprites ficam grudados e a segunda nota não é distinguível); a 2

frames é aviso (muito rápido); acordes de 3 ou mais notas e notas com mais de 15 frames de som geram aviso.

**Por que usar** `Fraction`**?** Com ponto flutuante, `1/3 * 6` poderia dar `1.9999999` e a verificação "a duração em frames é inteira?" falharia ou, pior, arredondaria errado. Com frações a conta é exata.

## 10.3 gera_charmap.py

Desenha os sprites do jogo nos códigos 1 a 31 do charmap original. Cada sprite é escrito no próprio script como uma matriz 8x8 de `#` (aceso) e `.` (apagado), o que deixa o desenho visível no código.

| Função | O que faz |
|---|---|
| `SPRITES` | dicionário código -> (nome, matriz) |
| geração da meia linha | cria os códigos 21 a 26 a partir das notas 1 a 3: `MEIA_CIMA` = 4 linhas vazias + as 4 de cima da nota; `MEIA_BAIXO` = as 4 de baixo + 4 vazias. Mudar o desenho da nota muda os seis sprites junto |
| `valida()` | recusa códigos fora de 1..31, o código 4 (não desenhado pelo simulador) e matrizes que não sejam 8x8 |
| `bloco_caracteres_controle()` | gera as 256 linhas (32 caracteres x 8) dos códigos 0..31, agrupando faixas vazias como `[a..b] : 00000000;` |
| `main()` | substitui o bloco de caracteres de controle do original e **preserva a quebra de linha** do arquivo de entrada |

**Por que preservar CRLF?** O `load_charmap` do simulador usa `fseek(-1)` em modo texto. No Windows isso só funciona com quebras CRLF; com LF, o charmap seria lido deslocado.

## 10.4 versao_sound.py e mede_sincronia.py

`versao_sound.py` gera `testes/jogo_sound.asm`, uma cópia do jogo com a `sndn` trocada por `sound`. O script exige que haja exatamente uma substituição, para garantir que a comparação é justa (a única diferença entre as versões é a instrução). Serve para demonstrar o problema: com `SOUND`, a tela e o teclado congelam a cada nota.

`mede_sincronia.py` roda o jogo no harness, no nível 5, e lê o log de notas (ciclo de cada `SNDN` e a duração pedida). Para cada nota, compara a duração pedida com o tempo real até a próxima nota, `(ciclo_seguinte - ciclo) / 5000` ms. Se as duas coisas batem, cada nota termina exatamente quando a seguinte começa.

# 11. Testes automáticos

## 11.1 Harness headless

`tools/headless/headless.cpp` é um simulador **sem janela** que usa o **mesmo** `Model.cpp` do simulador gráfico, com a mesma semântica de instruções. Os componentes de interface (vídeo, painel de registradores, controlador) são substituídos por stubs vazios. O `sound_stub.cpp` substitui o `Sound.cpp`: em vez de tocar, registra cada nota (ciclo, frequência, duração, onda e se foi bloqueante) e soma o tempo que a CPU ficaria travada pelas notas `SOUND`.

| Opção | Efeito |
|---|---|
| `--cycles N` | executa no máximo N instruções |
| `--key CICLO:TECLA` | injeta uma tecla a partir de um ciclo (simula o jogador) |
| `--dump CICLO` | imprime a tela 40x30 como texto naquele ciclo |
| `--autoplay` | liga o "robô" que joga sozinho |
| `--autoplay-linha N` | o robô aperta quando a nota está na linha N (para testar a janela) |
| `--colors`, `--mem`, `--glyphs` | grade de cores, memória e sprites do charmap |
| `--tempo-real S` | roda S segundos com o laço de clock fixo do simulador (para medir o clock) |
| `--gui-lock` | envolve cada instrução em `gdk_threads_enter/leave`, como o simulador gráfico |

Na tela em texto, os sprites aparecem como caracteres: `(=)` é uma nota, `[_]` um alvo, `:` a divisória, `*` a faísca, `{O}` o alvo preenchido, e `/=\` sobre `\=/` é uma nota em meia linha.

**O robô.** A cada 2000 ciclos, o robô olha a linha observada no centro de cada pista. Se vê o meio de uma nota (sprite 2) e nenhuma outra nota mais abaixo na mesma pista, aperta a tecla da pista. Ele só aperta se viu a mesma situação **em duas leituras seguidas**, para ignorar o instante em que o `MoveNotas` apagou uma nota e ainda não a redesenhou. Como ele lê a tela, e não a memória, testa o jogo do ponto de vista do jogador.

**Por que testar sem janela?** O teste fica reproduzível (as teclas entram em ciclos exatos), roda muito mais rápido que o tempo real e não depende de placa de som nem de interface gráfica.

## 11.2 Cenários do testa_jogo.py

Os cenários de pontuação rodam no nível 9, o mais rápido, para o teste terminar logo, e conferem a caixa FIM DE JOGO na última tela:

| Cenário | Resultado esperado |
|---|---|
| robô acerta tudo na linha 25 | 63/63, 1920 pontos, combo 63, 100%, conceito S |
| robô acerta na linha 24 ou 26 | idem (ainda PERFEITO) |
| robô acerta na linha 23 ou 27 | 63/63, 960 pontos (QUASE) |
| robô aperta na linha 21 | 0 acertos, conceito D: mesmo lida no frame seguinte, a nota estaria na 22, fora da janela |
| ninguém joga | 0 acertos, conceito D |
| cada música extra do menu (tecla M) | o robô acerta todas as notas; vale para qualquer partitura nova |
| tecla sem nota | alvo preenchido `{O}`, sem faísca, ERROS = 001 e, depois, o alvo volta ao normal |
| animação | aparecem notas inteiras `(=)` e em meia linha (`/=\` sobre `\=/`) |
| reinício | ESPAÇO no fim volta ao menu e a nova partida começa zerada |

Em todos os cenários de pontuação, o teste exige também **62 notas tocadas, todas via** `SNDN`**, 0 via** `SOUND` **e 0 ms de CPU travada**.

# 12. Simple Simulator

`SimpleSimulator/simple_simulator.c` implementa o Processador ICMC **ciclo a ciclo**, como no hardware, a partir do template da disciplina. Em vez de executar cada instrução numa linha de C (como o `Model.cpp`), ele modela o caminho de dados: registradores, multiplexadores, ULA, memória e os sinais de controle gerados por uma máquina de estados.

## 12.1 Estrutura de um ciclo

Cada volta do laço principal é um ciclo de clock e tem três fases:

1. **Borda de clock:** os registradores com sinal de carga ligado recebem o valor (`if (LoadIR) IR = DATA_OUT;`, `if (LoadReg[rx]) reg[rx] = M2;`, `if (RW == 1) MEMORY[M1] = M5;`...). Depois todos os sinais de carga são zerados.
2. **Unidade de controle:** um `switch(state)` liga os sinais de controle e escolhe as entradas dos muxes para o próximo ciclo, além de decidir o próximo estado.
3. **Lógica combinacional:** calcula as saídas dos muxes, na ordem das dependências (M4 antes de M1 e M2; leitura da memória antes de M2; M3 antes da ULA e de M5), e a ULA.

| Mux | Seleciona | Uso |
|---|---|---|
| M1 | PC, MAR, M4, SP | endereço da memória |
| M2 | ULA, DATA_OUT, M4, SP, teclado | dado escrito no registrador |
| M3 | r0..r7 ou FR (8) | operando X da ULA e dado a gravar |
| M4 | r0..r7 ou a constante 1 (8) | operando Y da ULA, endereço indireto, novo SP |
| M5 | PC ou M3 | dado escrito na memória |
| M6 | ULA ou DATA_OUT | novo valor de FR |

## 12.2 Máquina de estados

| Estado | O que faz |
|---|---|
| RESET | zera registradores e flags; `PC = 0`, `SP = 32767` |
| FETCH (busca) | `selM1 = sPC, RW = 0, LoadIR = 1, IncPC = 1` (comum a todas as instruções) |
| DECODE | decodifica o opcode e termina as instruções de 1 ciclo (LOADN, LOADI, STOREI, MOV, ULA, INC, CMP, JMP, PUSH...) |
| EXECUTE | 2º ciclo de LOAD, STORE, CALL, POP e RTS |
| EXECUTE2 | 3º ciclo do RTS: `IncPC`, para pular a palavra de endereço do CALL |
| HALTED | parado; `r` reinicia, q sai |

A implementação foi conferida contra os diagramas de estados da disciplina (Figma "CPU_ICMC-team-library"), instrução por instrução: busca, NOOP, LOADN, LOAD, STORE, LOADI, STOREI, MOV, ADD e demais operações da ULA, INC/DEC, CMP, JMP, CALL, RTS, PUSH e POP seguem os mesmos sinais de controle e as mesmas transições de estado. A única diferença é no MOV: o diagrama testa `IR0` e depois `IR1`, e o código usa `switch(IR[1..0])` (`00` -> Rx<-Ry, `01` -> Rx<-SP, demais -> SP<-Rx). Os dois só divergem na codificação `10`, que o montador nunca gera: para `MOV SP, Rx` ele escreve `11` (`montador.c`, linha 593). O simulador gráfico faz igual ao código.

## 12.3 Testes do Simple Simulator

`testes/simples/TesteCPU.asm` imprime uma letra por instrução testada (saída esperada `ABCDEFGHIJKLMNOP`). `testes/simples/TesteExtra.asm` cobre o que o jogo usa e o TesteCPU não cobre: `MOD`, `PUSH FR`/`POP FR`, `MOV SP`, chamadas aninhadas e `jle`/`jeg`/`jel` (saída esperada `abcdefghij`, idêntica à do simulador gráfico).

## 12.4 Limitações conhecidas da ULA do Simple Simulator

Os diagramas definem a máquina de controle, não os cálculos da ULA. Na revisão do código, estes pontos da ULA foram identificados como divergentes de uma ULA de 16 bits correta. O jogo não depende deles:

- `ADD` com estouro faz `result -= 65535` em vez de cortar em 16 bits (`0xFFFF + 1` dá 1 em vez de 0).
- `SUB` usa `unsigned`, então o teste `result < 0` nunca é verdadeiro e o flag de negativo não liga; o SUBC (bit 0) é ignorado.
- `MULT` e os deslocamentos não cortam o resultado em 16 bits.
- `rotl16`/`rotr16` usam `16*8` (128) em vez de 16 bits.
- O teste de limite de memória usa `M1 > TAMANHO_MEMORIA` em vez de `>=`.
- Os `#define` dizem `NEGATIVE = 9` e `DIV_BY_ZERO = 6`, mas a tabela de condições de salto testa os bits trocados (herdado do template).

# 13. Como compilar e executar

**Dependências.** Linux (Debian/Ubuntu): `build-essential pkg-config libgtk2.0-dev libsdl2-dev python3`. Windows: MSYS2 UCRT64 com `gcc`, `gtk2`, `SDL2` e `pkgconf`.

```
./build.sh                                  # montador, simulador, Simple Simulator e harness
./Montador/montador jogo.asm cpuram.mif     # monta o jogo
./Simulador/sim cpuram.mif charmap.mif      # roda (Home = execução automática)
python tools/testa_jogo.py                  # testes automáticos
```

`build.sh` detecta o Windows pelo `uname` e acrescenta a extensão `.exe` e a biblioteca `winmm` (usada pelo `timeBeginPeriod`).

**Adicionar uma música:**

```
python tools/gera_musica.py --validar tools/musicas/minha_musica.txt
python tools/gera_musica.py jogo.asm tools/musicas/ode_alegria.txt \
       tools/musicas/arpejo_la_menor.txt tools/musicas/minha_musica.txt
./Montador/montador jogo.asm cpuram.mif
python tools/testa_jogo.py
```

Todas as músicas que devem aparecer no menu precisam ser passadas: o bloco é regerado por inteiro, e a música que ficar de fora some do jogo.

**Alterar telas ou sprites:**

```
python tools/gera_telas.py jogo.asm tools/telas/jogo.txt tools/telas/menu.txt tools/telas/fim.txt
python tools/gera_charmap.py <charmap original> charmap.mif
```

# 14. Limites e restrições do projeto

| Limite | Valor | Origem |
|---|---|---|
| notas simultâneas na tela | 16 | slots de `notaY`/`notaPista` |
| duração do som de uma nota | 15 frames | campo de 4 bits no mapa |
| faixa de alturas | C3 a D8 (63 notas) | índice de 6 bits |
| sons simultâneos | 1 (monofônico) | semântica da `SNDN`; num acorde só a 1ª nota soa |
| notas por música no placar final | 999 | `ImprimeNumero` com 3 dígitos |
| pontuação | 65535 | palavra de 16 bits (placar de 5 dígitos) |
| precisão | até 655 acertos | `acertos * 100` em 16 bits |
| tamanho de uma música | ~1 palavra por frame | a memória de 32768 palavras comporta músicas de vários minutos |
| nome no menu / painel | 20 / 3 x 9 caracteres | largura fixa das áreas na tela |